

Rising Waters: Machine Learning-Based Flood Prediction System

Project Description:

Rising Waters is a machine learning-based flood prediction system developed to assist in the early identification of flood-prone conditions using historical weather and rainfall data. The project analyses important environmental parameters such as annual rainfall, seasonal rainfall, cloud cover, river discharge, historical flood records, elevation, and infrastructure-related factors to predict the likelihood of flooding. By utilizing machine learning algorithms, the system provides timely and reliable predictions that can support disaster preparedness and risk management.

The application is built using Python and Flask, with data preprocessing and model development carried out using Jupyter Notebook. Several machine learning algorithms, including Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost, were trained and evaluated. The best-performing model was saved and integrated into a Flask web application that allows users to enter flood-related parameters and instantly receive a flood risk prediction through a simple and user-friendly web interface.

Scenarios

Scenario 1:

A disaster management officer enters weather parameters such as annual rainfall, seasonal rainfall, cloud cover, and river discharge into the application. The system processes the input using the trained machine learning model and predicts whether the region has a high or low risk of flooding, enabling authorities to take precautionary measures in advance.

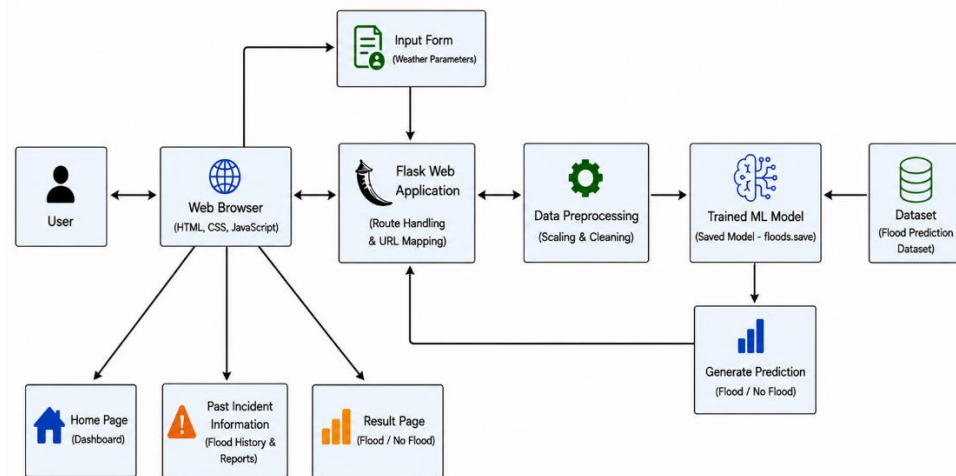
Scenario 2:

A local government official analyses rainfall and environmental conditions for a particular district before the monsoon season. Based on the prediction generated by the system, the administration can identify vulnerable areas and prepare emergency response plans, evacuation strategies, and resource allocation.

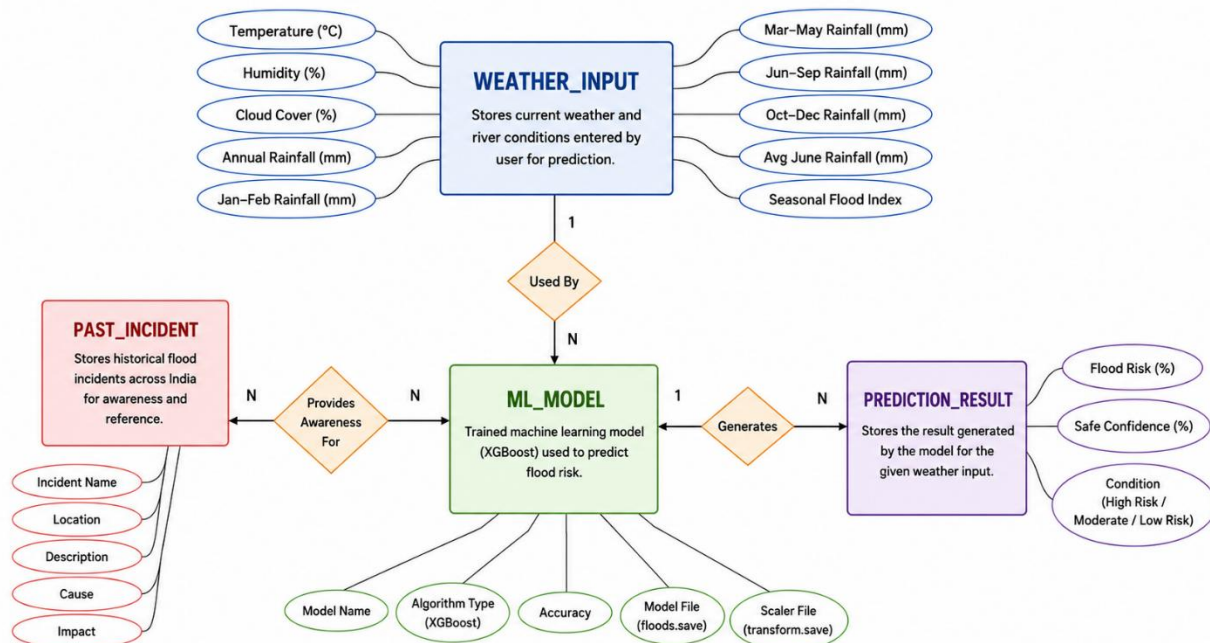
Scenario 3:

A researcher or environmental analyst provides historical weather and flood-related data to evaluate flood risk patterns. The application predicts flood occurrence using the trained model, helping researchers compare results, study climate trends, and improve future flood prediction strategies.

Technical Architecture:



ER Diagram:



Description:

Rising Waters follows a modular multi-layer architecture to provide efficient and accurate flood prediction. The presentation layer offers an interactive web interface developed using HTML, CSS, and JavaScript, allowing users to enter weather-related parameters. The Flask-based application layer validates the input data, performs preprocessing using the saved scaler and transformer objects, loads the trained machine learning model, and generates flood predictions. The machine learning layer contains the trained prediction model and preprocessing objects, while the data layer consists of the historical flood dataset used for data preprocessing, feature engineering, model training, and evaluation. This architecture ensures accurate predictions, modular development, and easy maintenance.

Pre-requisites:

- Python Programming Knowledge: Python 3.8 or above
- Machine Learning Basics: Scikit-learn, Decision Tree, Random Forest, KNN, XGBoost
- Flask Framework Knowledge: Flask Documentation
- HTML, CSS and JavaScript Skills
- Data Analysis Libraries: NumPy, Pandas, Matplotlib, Seaborn
- Jupyter Notebook / Anaconda Navigator
- Version Control with Git (Optional)
- Basic Knowledge of Machine Learning Model Deployment using Flask.

Project Workflow:

Milestone 1: Environment Setup

- **Activity 1.1:** Set up the Python development environment using Anaconda Navigator and Jupyter Notebook.
- **Activity 1.2:** Install all required libraries including NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn, Flask, and Joblib/Pickle.
- **Activity 1.3:** Configure the project structure and verify all dependencies for machine learning and web application development.

Milestone 2: Dataset Collection

- **Activity 2.1:** Collect the flood prediction dataset from Kaggle containing historical weather and rainfall information.
- **Activity 2.2:** Study the dataset attributes such as annual rainfall, cloud visibility, seasonal rainfall, and other meteorological parameters.
- **Activity 2.3:** Import the dataset into Jupyter Notebook for further analysis and preprocessing.

Milestone 3: Data Visualization & Analysis

- **Activity 3.1:** Perform univariate analysis to understand the distribution of individual features.
- **Activity 3.2:** Conduct multivariate analysis to identify relationships between different weather parameters.
- **Activity 3.3:** Generate descriptive statistics, box plots, distribution plots, and heat maps to identify trends, feature correlations, and outliers.

Milestone 4: Data Preprocessing

- **Activity 4.1:** Handle missing values and remove duplicate records from the dataset.
- **Activity 4.2:** Detect and treat outliers to improve data quality.
- **Activity 4.3:** Encode categorical values (if applicable) and separate dependent and independent variables.
- **Activity 4.4:** Apply feature scaling and split the dataset into training and testing sets.

Milestone 5: Machine Learning Model Development

- **Activity 5.1:** Train multiple machine learning algorithms including Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost.
- **Activity 5.2:** Evaluate model performance using the Confusion Matrix, Classification Report, and Accuracy Score.
- **Activity 5.3:** Select the best-performing model based on prediction accuracy and save the trained model as floods.save along with the preprocessing objects.

Milestone 6: Flask Web Application Development

- **Activity 6.1:** Develop the Flask backend to load the trained machine learning model and process user inputs.
- **Activity 6.2:** Design responsive web pages including the Home Page, Prediction Input Page, Flood Chance Result Page, and No Flood Chance Result Page using HTML, CSS, and JavaScript.
- **Activity 6.3:** Integrate the Flask backend with the frontend to provide real-time flood prediction results.

Milestone 7: Application Execution

- **Activity 7.1:** Install all the required Python libraries listed in the requirements.txt file before running the application.
- **Activity 7.2:** Launch the flask application.
- **Activity 7.3:** Open the browser for running the application.

Milestone 1: Environment Setup

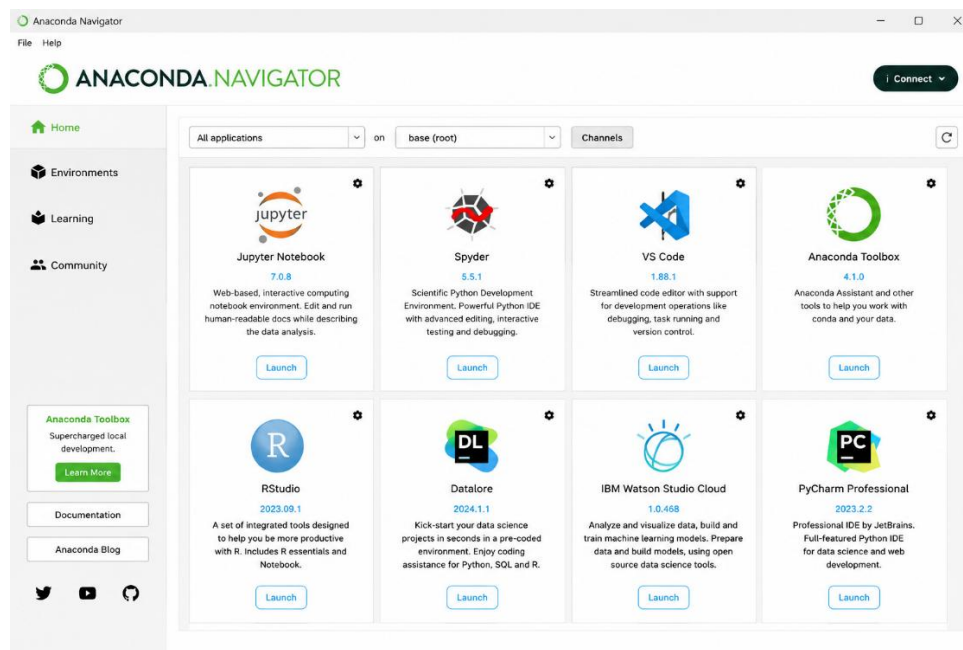
In this milestone, we prepare the development environment required for building the **Rising Waters: Machine Learning-Based Flood Prediction System**. The objective is to install the necessary software, configure the Python environment, and verify that all required machine learning and web development libraries are available before beginning dataset analysis and model development.

Activity 1.1: Set Up the Development Environment

Before starting the project, install Anaconda Navigator and create a Python environment for developing and testing the flood prediction system.

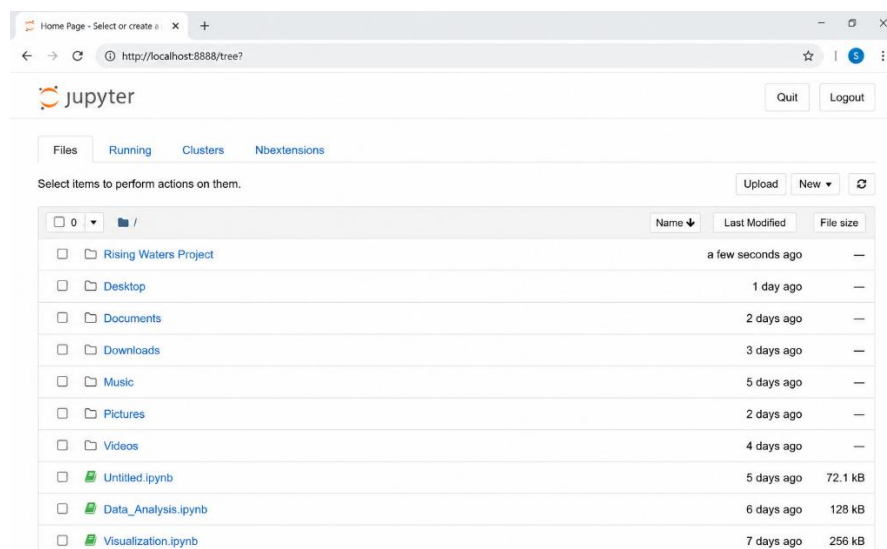
Step 1 — Install Anaconda Navigator

Install **Anaconda Navigator**, which provides a unified environment for managing Python packages and development tools required for the project. And launch the **Jupyter Notebook**.



Step 2 — Jupyter Notebook

After launching **Jupyter Notebook** open the notebook. Create a new Python notebook to begin dataset analysis and machine learning model development. And name it to Rising Waters Project



Activity 1.2: Install and Verify Required Libraries

Step 3 – Import Required Libraries

Import all the required Python libraries into Jupyter Notebook to verify that the development environment has been configured successfully.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score
)

import joblib
import pickle

from flask import Flask, render_template, request
```

Activity 1.3: Configure the Project Structure and Verify Dependencies

A well-organized project structure was created to separate the machine learning components, web application files, dataset, and supporting resources. This modular organization improves code readability, maintainability, and simplifies future enhancements.

The project directory consists of the following components:

- **Dataset/** – Stores the flood prediction dataset (Flood_Prediction_Dataset.csv) used for training and testing the machine learning models.
- **static/** – Contains static resources such as images, CSS, and JavaScript files required for the web interface.
- **templates/** – Includes all HTML pages used by the Flask application, such as the Home Page, Prediction Page, Flood Chance Result Page, and No Flood Chance Result Page.
- **Flood_Prediction_Code.ipynb** – Jupyter Notebook containing data preprocessing, visualization, model training, evaluation, and model selection.
- **app.py** – Flask application responsible for handling user requests, loading the trained model, processing input parameters, and generating prediction results.
- **floods.save** – Stores the trained machine learning model selected for flood prediction.
- **scaler.pkl** – Stores the feature scaling object used during preprocessing to ensure consistent prediction.
- **transform.save** – Stores the preprocessing transformation object used before model prediction.
- **requirements.txt** – Lists all Python libraries and project dependencies required to execute the application successfully.

The project structure and all required dependencies were verified to ensure seamless integration between the machine learning model and the Flask-based web application.

```
Rising_Waters_Project/  
|  
├── Dataset/  
│   └── Flood_Prediction_Dataset.csv  
|  
├── static/  
│   ├── Images/  
│   ├── main.css  
│   └── main.js  
|  
├── templates/  
│   ├── home.html  
│   ├── index.html  
│   ├── chance.html  
│   └── no_chance.html  
|  
├── Flood_Prediction_Code.ipynb  
├── app.py  
├── floods.save  
├── requirements.txt  
├── scaler.pkl  
└── transform.save
```

Milestone 2: Dataset Collection

This milestone focuses on collecting a reliable flood prediction dataset and understanding its features before building the machine learning models. The dataset serves as the foundation for data analysis, preprocessing, and flood prediction.

Activity 2.1: Collect the Flood Prediction Dataset

The flood prediction dataset was collected from the Kaggle platform, which provides publicly available datasets for machine learning research. It contains historical weather and rainfall information required for flood prediction.

flood dataset.xlsx (16.03 kB)

Download

Open in new window

More options

About this file

Suggest Edits

This file does not have a description yet.

Table	Total Rows	Total Col...
Sheet5	115	11

Activity 2.2: Study the Dataset Attributes

The dataset was renamed to Flood_Prediction_Dataset and converted to csv file and it is examined to understand important features such as annual rainfall, cloud visibility, seasonal rainfall, and other meteorological parameters.

First Five Records:											
	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
0	29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.8667	649.9	0
1	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.3000	256.4	1
2	28	75	42	3271.2	21.7	336.3	2343.0	570.1	186.2000	308.9	0
3	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.0667	862.5	0
4	31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.4000	586.9	0

Activity 2.3: Import the Dataset into Jupyter Notebook

The dataset was imported into Jupyter Notebook using the Pandas library for further preprocessing and analysis. After loading, the dataset was verified to ensure it was successfully read into the notebook.

```
df = pd.read_csv("Dataset/Flood_Prediction_Dataset.csv")

print("First Five Records:")
display(df.head())
```

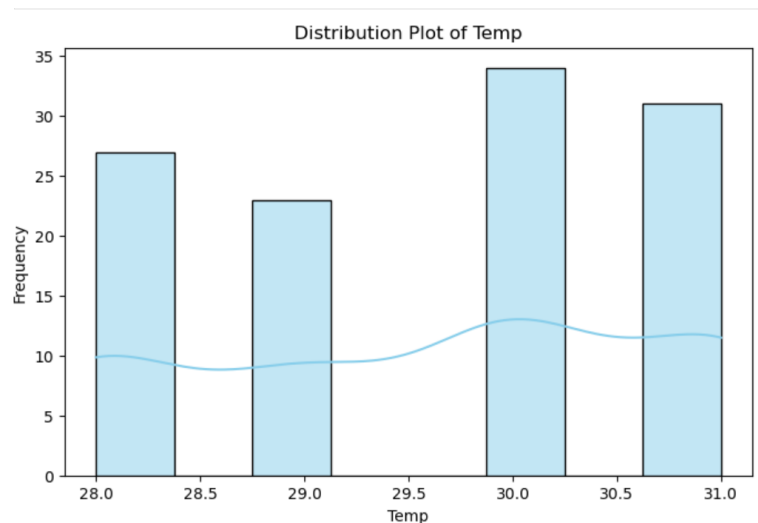
Milestone 3: Data Visualization & Analysis

This milestone focuses on exploring the flood prediction dataset through various visualization and statistical techniques. Data analysis helps identify feature distributions, relationships, trends, and anomalies before model training.

Activity 3.1: Perform Univariate Analysis

Univariate analysis was performed to examine the distribution of individual features such as rainfall and weather parameters. This helped understand the characteristics and variability of each feature independently.

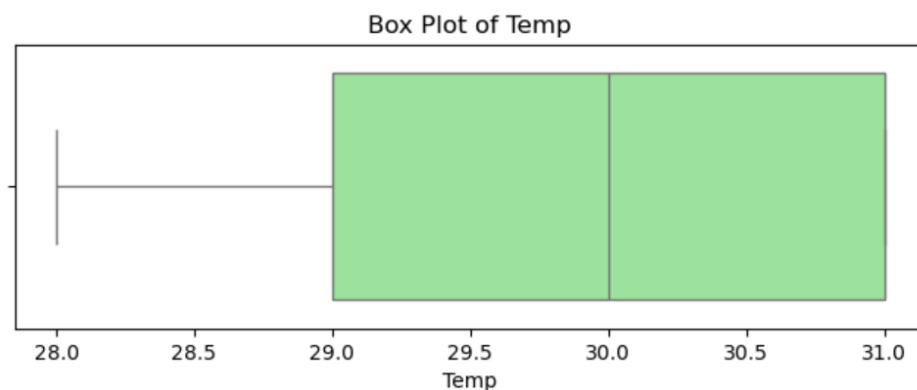
Example:



Activity 3.2: Conduct Multivariate Analysis

Multivariate analysis was carried out to study the relationships between multiple weather-related features. This analysis helps identify patterns and dependencies that influence flood prediction.

Example:



Activity 3.3: Generate Descriptive Statistics and Visualizations

Descriptive statistics, box plots, distribution plots, and heat maps were generated to summarize the dataset, identify feature correlations, and detect potential outliers. These visualizations improve data understanding before preprocessing.

Statistical Summary:

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
count	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000
mean	29.600000	73.852174	36.286957	2925.487826	27.739130	377.253913	2022.840870	497.636522	218.100871	439.801739	0.139130
std	1.122341	2.947623	4.330158	422.112193	22.361032	151.091850	386.254397	129.860643	62.547595	210.438813	0.347597
min	28.000000	70.000000	30.000000	2068.800000	0.300000	89.900000	1104.300000	166.600000	65.600000	34.200000	0.000000
25%	29.000000	71.000000	32.500000	2627.900000	10.250000	276.750000	1768.850000	407.450000	179.666700	295.000000	0.000000
50%	30.000000	74.000000	36.000000	2937.500000	20.500000	342.000000	1948.700000	501.500000	211.033300	430.600000	0.000000
75%	31.000000	76.000000	40.000000	3164.100000	41.600000	442.300000	2242.900000	584.550000	263.833300	577.650000	0.000000
max	31.000000	79.000000	44.000000	4257.800000	98.100000	915.200000	3451.300000	823.300000	366.066700	982.700000	1.000000

Milestone 4: Data Preprocessing

This milestone focuses on preparing the dataset for machine learning by cleaning, transforming, and organizing the data. Proper preprocessing improves data quality and enhances the accuracy of the flood prediction models.

Activity 4.1: Handle Missing Values and Remove Duplicate Records

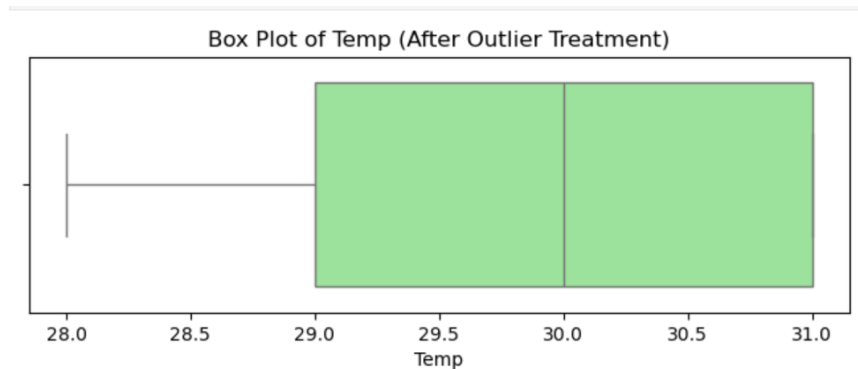
The dataset was examined for missing values and duplicate records to ensure data consistency and reliability. Missing values were handled appropriately, and duplicate entries were removed before further analysis.

Missing Values in Each Column:

```
Temp          0
Humidity      0
Cloud Cover   0
ANNUAL        0
Jan-Feb       0
Mar-May       0
Jun-Sep       0
Oct-Dec       0
avgjune       0
sub           0
flood         0
dtype: int64
```

Activity 4.2: Detect and Treat Outliers

Outliers were identified using box plots and statistical techniques to detect abnormal values in the dataset. Appropriate preprocessing methods were applied to minimize their impact on model performance.



Activity 4.3: Encode Categorical Values and Separate Features

Categorical variables were encoded into numerical values suitable for machine learning algorithms. But in the dataset there are no Categorical variables. The dataset was then divided into independent features (X) and the target variable (y).

```
No categorical columns found.  
No Label Encoding or Feature Mapping is required.
```

Activity 4.4: Apply Feature Scaling and Split the Dataset

Feature scaling was performed using StandardScaler to normalize the input features before model training. Finally, the dataset was divided into training and testing sets using the train-test split method.

```
Training Feature Set : (92, 10)  
Testing Feature Set  : (23, 10)  
Training Target Set  : (92,)  
Testing Target Set   : (23,)
```

Milestone 5: Machine Learning Model Development

This milestone focuses on training multiple machine learning algorithms and evaluating their performance to identify the most accurate flood prediction model. The selected model is then saved for deployment in the Flask web application.

Activity 5.1: Train Machine Learning Models

Multiple machine learning algorithms, including Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost, were trained using the prepared dataset. Each model learned the relationship between weather parameters and flood occurrence.

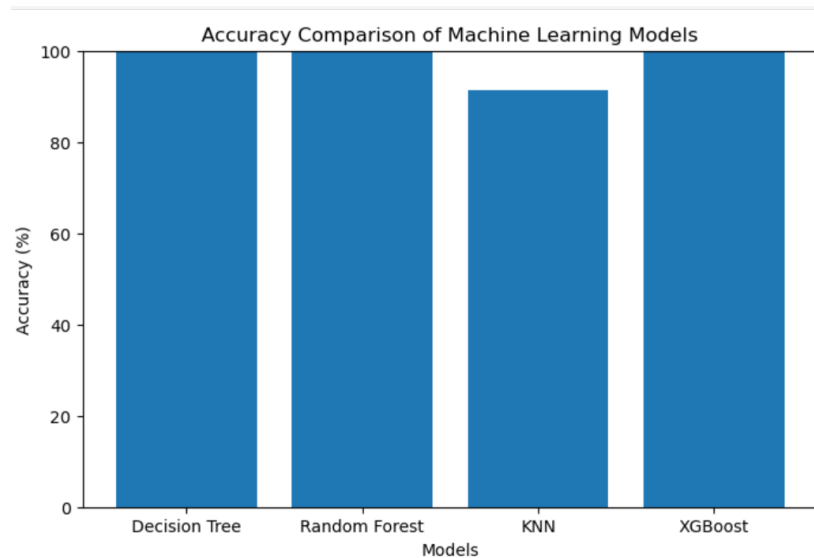
	Model	Accuracy (%)
0	Decision Tree	100.000000
1	Random Forest	100.000000
2	KNN	91.304348
3	XGBoost	100.000000

Activity 5.2: Evaluate Model Performance

The trained models were evaluated using the Confusion Matrix, Classification Report, and Accuracy Score. These evaluation metrics helped compare the performance of each model and determine the most suitable algorithm.

```
print("Final Selected Model : XGBoost")  
  
print("Accuracy :", round(xgb_accuracy * 100, 2), "%")
```

```
Final Selected Model : XGBoost  
Accuracy : 100.0 %
```



Activity 5.3: Select and Save the Best Model

Based on the evaluation results, the best-performing model XGBoost was selected. The trained model was saved as **floods.save**, and the preprocessing objects such as the scaler and transformer were also saved for future predictions.

```
import joblib

joblib.dump(xgb_model, "floods.save")

joblib.dump(sc, "transform.save")

print("Model saved as floods.save")
print("Scaler saved as transform.save")
```

```
Model saved as floods.save
Scaler saved as transform.save
```

Milestone 6: Flask Web Application Development

This milestone focuses on developing a user-friendly web application using the Flask framework. The trained machine learning model is integrated with the frontend to provide real-time flood prediction based on user-provided weather parameters.

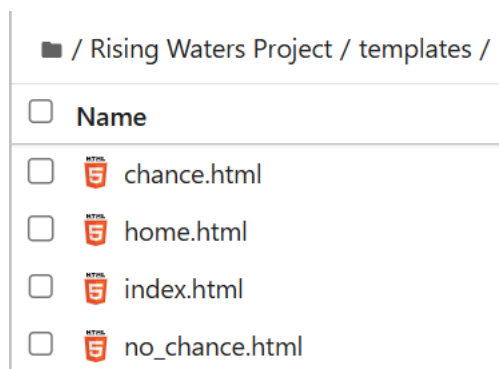
Activity 6.1: Develop the Flask Backend

The Flask backend was developed to load the trained machine learning model, process user inputs, and generate flood prediction results. It also handles routing, data preprocessing, and communication between the frontend and the prediction model.

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
model_path = os.path.join(BASE_DIR, "floods.save")
scaler_path = os.path.join(BASE_DIR, "transform.save")
load_error = None
```

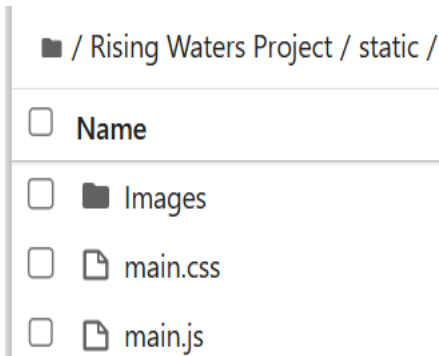
Activity 6.2: Design Responsive Web Pages

The user interface was developed using **HTML, CSS, and JavaScript** to provide an interactive and responsive experience. The application consists of the Home Page, Prediction Input Page, Flood Chance Result Page, and No Flood Chance Result Page.



Activity 6.3: Integrate the Backend with the Frontend

The Flask backend was integrated with the frontend to process user inputs and display prediction results instantly. When the user submits flood-related parameters, the backend loads the trained model, performs prediction, and returns the appropriate result page.



Milestone 7: Application Execution

This milestone explains how to execute the Flood Prediction application on a local system. It describes the steps required to launch the Flask server and access the web application through a browser.

Activity 7.1: Install Project Dependencies

Install all the required Python libraries listed in the requirements.txt file before running the application. This ensures that all dependencies are available for the Flask application.

```
pip install -r requirements.txt
```

Activity 7.2: Launch the Flask Application

Run the Flask application by executing the app.py file. Once the server starts successfully, the application becomes available through the local development server.

```
python app.py
```

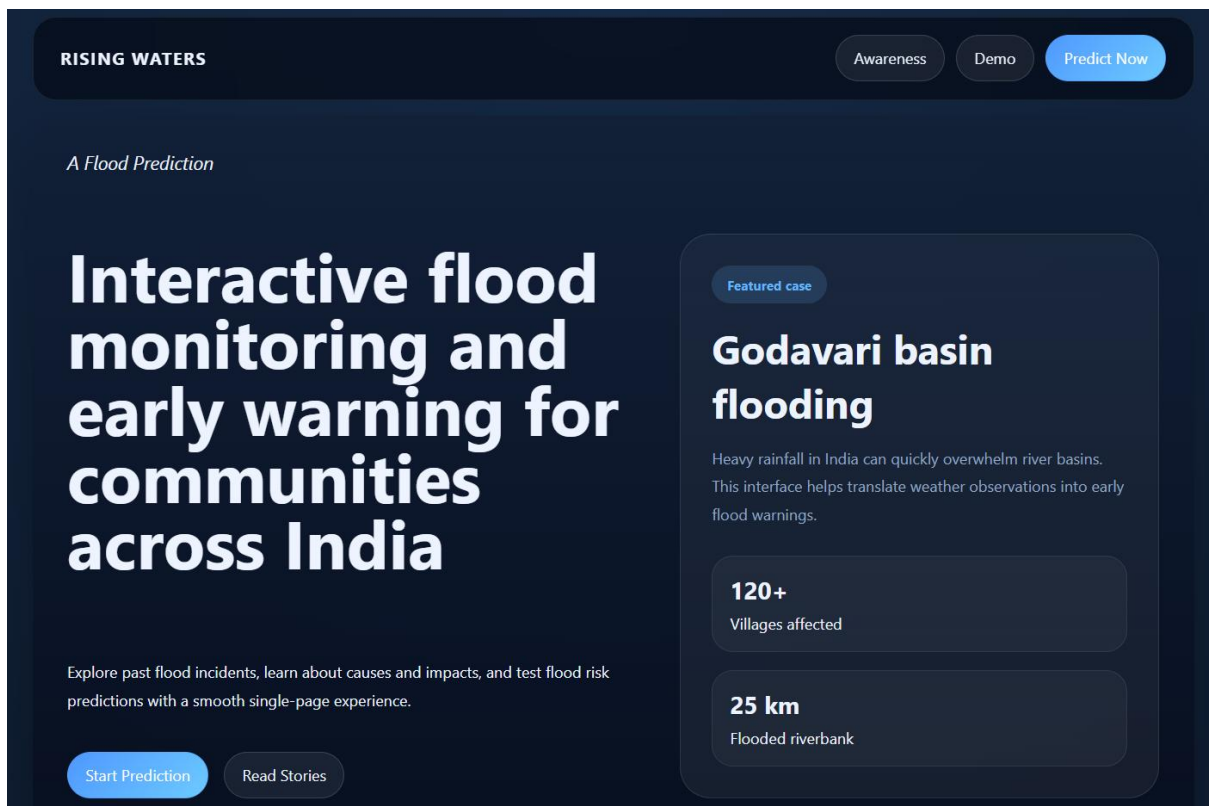
Activity 7.3: Access the Web Application

Open a web browser and navigate to the local server URL to access the Flood Prediction application. Users can now enter weather parameters and obtain flood prediction results through the web interface.

```
http://127.0.0.1:5000/
```

Exploring the Website's Web Pages:

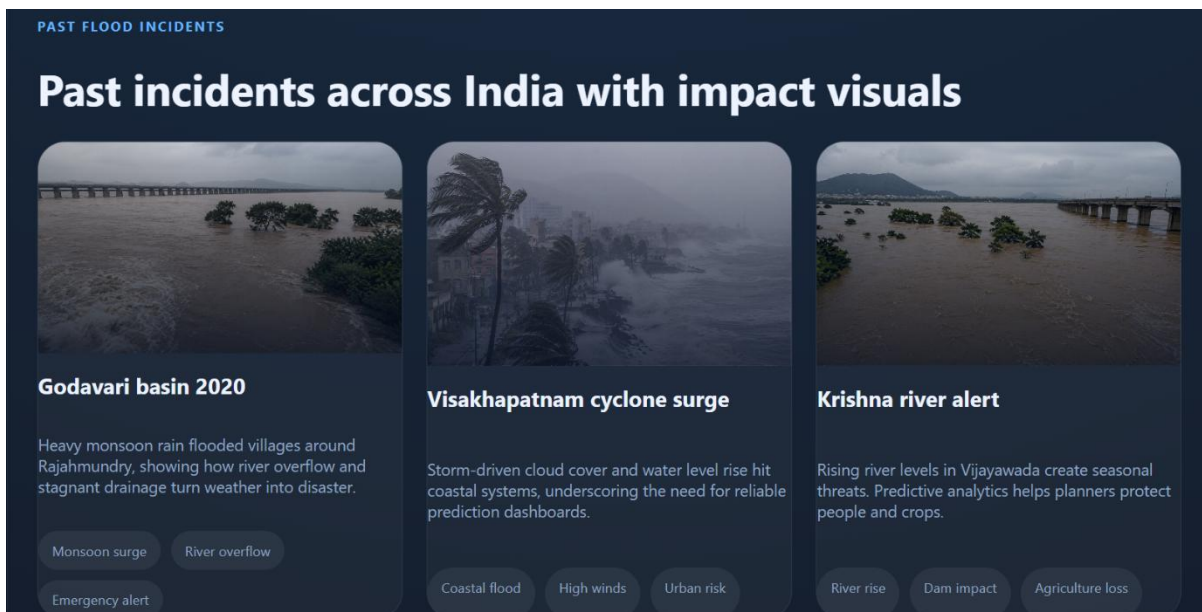
Home Page



The Home Page serves as the landing page of the Rising Waters flood prediction system. It introduces the purpose of the application by highlighting its role in interactive flood monitoring and early warning for communities across India. The navigation bar provides quick access to the Awareness, Demo, and Predict Now sections, enabling users to explore flood-related information and access the prediction system with ease.

The page also showcases a featured flood incident from the Godavari Basin, along with key statistics to raise awareness about the impact of flooding. Interactive buttons such as Start Prediction and Read Stories guide users to the flood prediction module and past incident information, providing an intuitive and user-friendly experience. The modern and responsive interface ensures easy navigation while emphasizing disaster awareness and preparedness.

Past Flood Incidents Page



The **Past Flood Incidents** page provides users with information about significant flood events that have occurred in different regions of India. It showcases real flood scenarios, including the **Godavari Basin Floods**, **Visakhapatnam Cyclone Surge**, and **Krishna River Floods**, along with representative images and brief descriptions of their impacts.

This section helps users understand how heavy rainfall, river overflow, cyclones, and changing weather conditions have affected communities in the past. It also highlights important factors such as **monsoon surge**, **river overflow**, **coastal flooding**, **high winds**, **dam impact**, and **agricultural loss**, creating greater awareness about flood risks. By presenting historical incidents, the application educates users on the importance of early flood prediction and preparedness, encouraging timely action to reduce the impact of future disasters.

Prediction Page

FLOOD PREDICTION

Enter current weather and river conditions

Live prediction result

Adjust the values and the dashboard will instantly show your safe score, flood risk, and final condition.

Flood risk

0%

Safe confidence

100%

CONDITION

Awaiting input

Fill in the form to generate an instant assessment with glowing chart visuals.

Temperature (°C)

28

Humidity (%)

75

Cloud Cover (%)

40

Annual Rainfall (mm)

3326.6

Jan-Feb Rainfall (mm)

9.3

Mar-May Rainfall (mm)

275.7

Jun-Sep Rainfall (mm)

2403.4

Oct-Dec Rainfall (mm)

638.2

Avg June Rainfall (mm)

130.3

Seasonal Flood Index

256.4

Check Now

Clear

The **Prediction Page** is the core module of the Rising Waters application, where users can predict the possibility of flooding by entering current weather and rainfall information. The interface allows users to provide parameters such as **Temperature**, **Humidity**, **Cloud Cover**, **Annual Rainfall**, **Seasonal Rainfall (Jan–Feb, Mar–May, Jun–Sep, Oct–Dec)**, **Average June Rainfall**, and **Seasonal Flood Index**.

After clicking the "**Check Now**" button, the entered values are processed by the trained machine learning model through the Flask backend. The application instantly displays the **Flood Risk Percentage**, **Safety Confidence**, and the final prediction indicating whether there is a **Flood Chance** or **No Flood Chance**. The **Clear** button resets all input fields, enabling users to perform multiple predictions quickly. This page provides a simple, interactive, and user-friendly interface for real-time flood risk assessment.

Results Page



The Prediction Result Page displays the final output generated by the trained machine learning model after analyzing the user-provided weather and rainfall parameters. The prediction results are presented through intuitive circular charts that show the Flood Risk Percentage and Safe Confidence Percentage, making it easy for users to understand the current flood situation.

In this example, the application predicts a Flood Risk of 92.45% with a Safe Confidence of 7.55%, indicating a very high probability of flooding under the given environmental conditions. The system classifies the situation as "High Risk" and displays an alert message advising users to follow safety procedures and share warnings with others. This real-time prediction helps authorities and the public make informed decisions, enabling early preparedness and reducing the potential impact of floods.

Conclusion

The **Rising Waters: A Machine Learning Approach to Flood Prediction** project successfully demonstrates how machine learning can be applied to support early flood risk assessment and disaster preparedness. By utilizing historical weather and rainfall data, the system analyzes multiple environmental parameters to predict the likelihood of flooding with high accuracy.

Throughout the project, various machine learning algorithms, including **Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost**, were trained and evaluated. The best-performing model was selected, saved, and integrated into a **Flask-based web application**, enabling users to obtain real-time flood predictions through a simple and interactive interface.

The web application not only predicts flood risk but also provides awareness through historical flood incidents and visual prediction results, making the system informative and easy to use. The project demonstrates the complete machine learning workflow, including data collection, preprocessing, visualization, model training, evaluation, deployment, and user interaction.

Overall, the proposed solution contributes to disaster management by providing timely flood predictions that can assist government authorities, disaster management agencies, and the public in making informed decisions. In the future, the system can be enhanced by integrating real-time weather APIs, IoT sensor data, satellite imagery, and Geographic Information Systems (GIS) to improve prediction accuracy and provide location-specific flood warnings on a larger scale.